

Sistemas Operativos: Práctica 1

Shaofan Xu Javier Maria Santa

March 1, 2026

1 Introducción

En esta práctica el objetivo es implementar códigos en C que dado una función Hash $\text{pow} : \mathbb{Z} \rightarrow \mathbb{Z}/9997697\mathbb{Z}$ con $x \mapsto (X \times x + y) \bmod 9997697$ y la imagen, debemos sacar su correspondiente x . Donde aprovechamos los conocimientos de procesos e hilos para simular la minería de un blockchain.

2 Requisitos cumplidos y no cumplidos

2.1 Requisitos cumplidos

Se han implementado correctamente los siguientes objetivos principales de la práctica:

- **Control de salida (a):** Impresión exacta de los mensajes de estado (*“Miner exited with status 0”*, *“Logger exited with status 0”*, etc.) según las especificaciones.
- **Gestión de hilos y concurrencia (b):** Como sabemos que el rango de la preimagen está en $[0, 9997697]$ (para simplificar el problema), creamos N hilos mediante `pthread_create` para dividir el espacio de búsqueda en N , para cada uno de los hilos se encarga de cada parte de espacio de búsqueda, una vez un hilo se ha encontrado la solución, los otros inmediatamente tienen que parar. Así, sucesivamente para cada ronda.
- **Gestión de procesos (c):** Creación del proceso hijo Registrador mediante `fork ()` a partir de un proceso padre que denominamos Minero, donde este se encarga de crear hilos para buscar de manera a fuerza bruta la preimagen (x) de una imagen dada (y). Y Registrador se encarga de guardar los resultados en un fichero.
- **Comunicación IPC (c):** Se realiza a través de tuberías (pipe), donde establecemos 2 tuberías: Minero a Registrador y Registrador a Minero, y cerrando los lados que no se usen.
 - Minero a Registrador (pipe_miner_reg): Para que el Minero manda a Registrador el resultado de cada ronda de búsqueda. Y el Registrador recibe los resultados e imprimen en un fichero “ppid.log” con el formato dado. Este proceso, se continúa hasta que el Minero manda una señal de terminación (-1 como solución en nuestro caso)

- Registrador a Minero (`pipe_reg_miner`): El Registrador manda una confirmación (1 en nuestro caso) de que se ha recibido correctamente por la tubería. Una vez el Minero recibe la confirmación se sigue con la siguiente ronda.

- **Control de errores:**

- **System Call** Hemos tenido en cuenta los casos en la que se usan llamadas al sistema como *malloc, pipe, pthread_create, etc*, así como sus controles de errores y liberación.
- **Gestión adecuada de hilos:** El proceso principal Minero espera a todos los hilos creados mediante `pthread_join()` antes de enviar los resultados de la ronda al Registrador.
- **Gestión adecuada de proceso:** Cuando termina todas las rondas el proceso principal Minero espera al proceso hijo Registrador mediante `wait(&status)`.
- **Gestión adecuada de tuberías:** Durante la ejecución de programas, cada uno de los procesos se cierran el lado de tubería que no lo van a utilizar, y una vez, alcanzado el máximo de las rondas se cierran todas las tuberías.

2.2 Requisitos no cumplidos

Todos los requisitos obligatorios especificados en el enunciado han sido implementados y funcionan correctamente.

3 Pruebas: Funcionamiento y Rendimiento

3.1 Pruebas de Funcionamiento

Se han ejecutado diversas pruebas para asegurar el correcto funcionamiento del programa.

```
./miner 100 50 4
```

3.2 Pruebas de Rendimiento